

Reinforcement Learning for Diffusion Models

RL for Real

Aviv Rosenberg
Google Research

October 14, 2026
NVIDIA Offices, Tel Aviv

Lecture Outline

01 Motivation & Applications

02 Foundations of Diffusion

03 The RL/MDP Formulation

04 RL Algorithms (RWR, DDPO, Flow-GRPO, Diffusion-DPO)

05 Discrete Diffusion Models & RL-D^2

06 Outlook & Conclusion

Section 01

Motivation & Applications



The Generative AI Boom

The Era of High-Fidelity Generation

We are witnessing an unprecedented boom in generative models, producing high-fidelity, photorealistic images, videos, and audio.

The Status Quo of Training

The standard training paradigm is straightforward: models are trained to mimic the underlying data distribution of massive datasets.

The Mechanism

Diffusion models learn a denoising process, starting from pure noise and iteratively refining it into a high-quality sample.



Denoising Process (Iterative Refinement)

NANO BANANA | STABLE DIFFUSION | VEO 3

Key Applications in Vision

Visual Alignment

Maximize aesthetic quality, lighting, and composition beyond average training data distribution.

Safety & Constraints

Penalize NSFW or biased outputs with a safety reward, so the model itself avoids them instead of relying only on prompt filters.



Standard (P-data)



RL-Aligned

Unsafe Sample → Penalized by a Safety Reward in Training

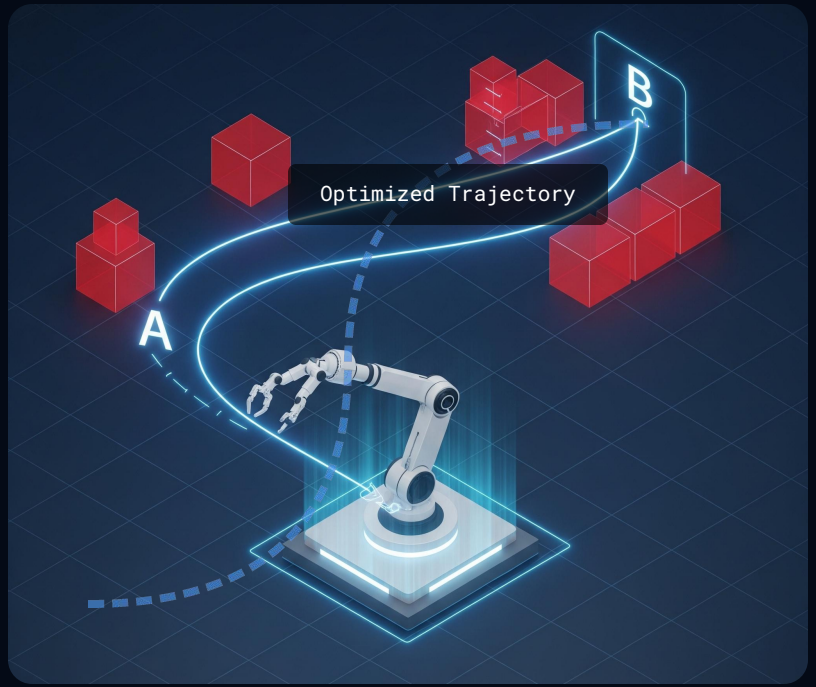
Key Applications in Robotics

Robotics & Trajectory Planning

Diffusion policies generate robot action trajectories; RL fine-tuning (e.g., DPPO) then maximizes task reward.

`+ Reward: Collision Avoided`

`+ Reward: Energy Minimized`



The Generative Alignment Problem (Part 1)

The Limitation of Mimicry

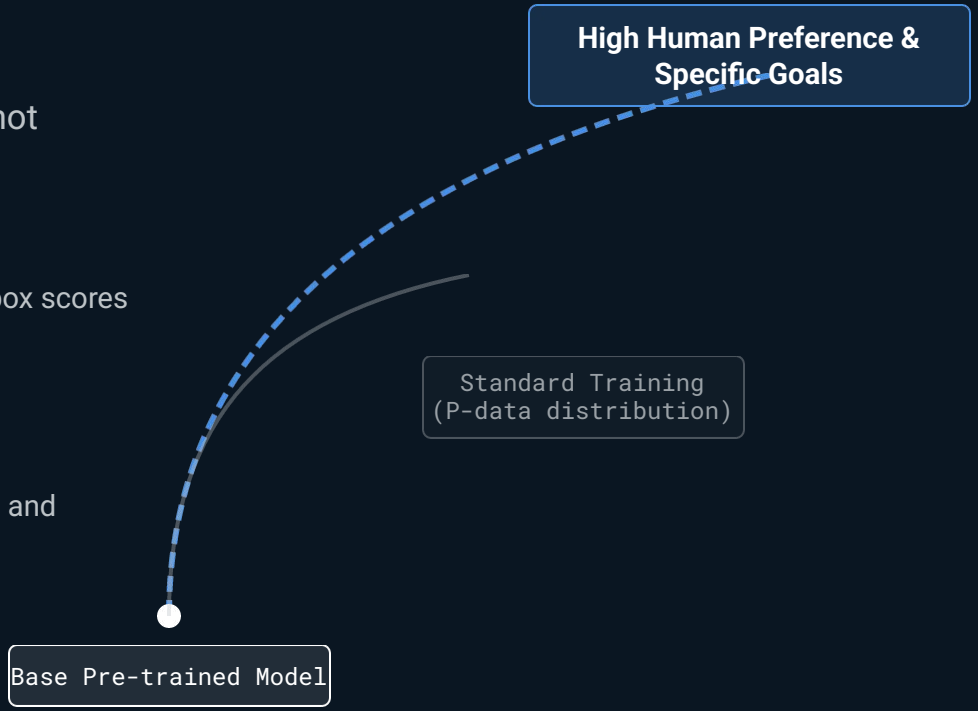
What happens when mimicking the data is not enough?

Objectives Beyond Likelihood

Human preference, prompt fidelity, safety, or black-box scores (OCR, detectors, VLM judges, file size). Many are non-differentiable; RL only needs reward values.

Alignment Gap

The divergence between maximizing data likelihood and optimizing for utility or safety.



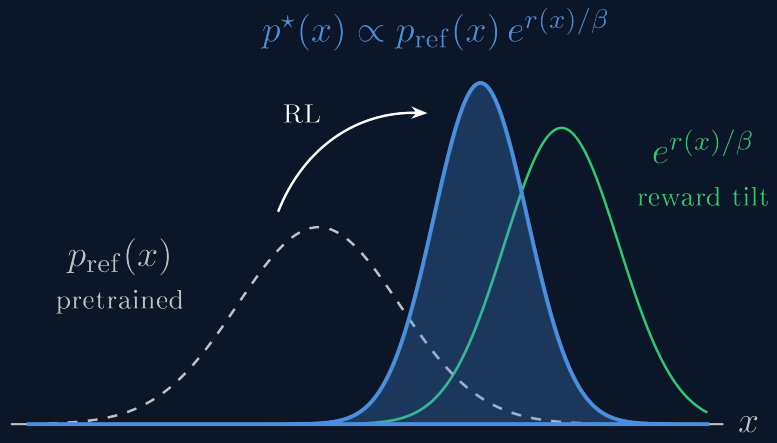
The Generative Alignment Problem (Part 2)

The Shift in Objective

Pre-training learns `P(data)`; we want **`P(data | high reward)`**.

The Bridge

RL tilts the pretrained model toward high reward while staying close to it (right). The temperature β sets the trade-off; $\beta \to 0$ collapses onto argmax r, the root of reward hacking.





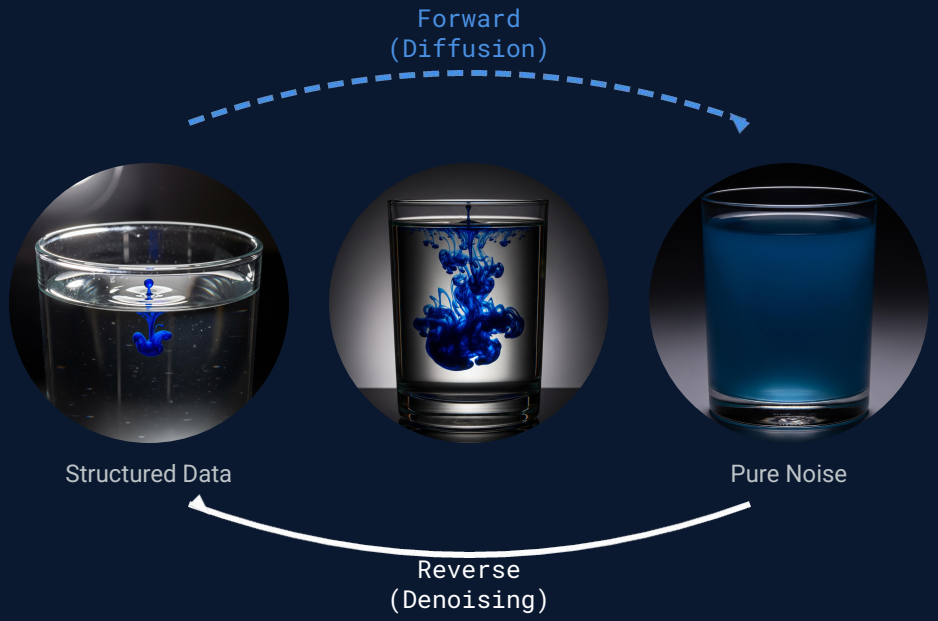
Section 02

Foundations of Diffusion

The Forward Process (Concept)

A fixed Markov chain that gradually adds Gaussian noise to data.

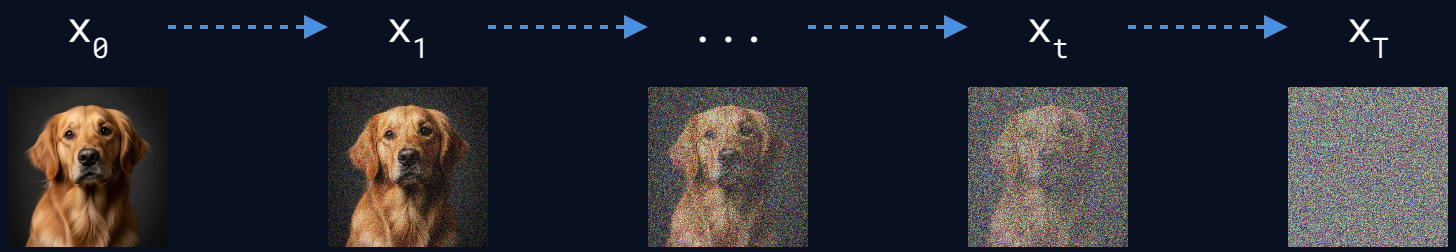
Over T steps, data becomes pure white noise.



The Forward Process (Math)

Single-step transition with a small variance schedule β_t:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right)$$



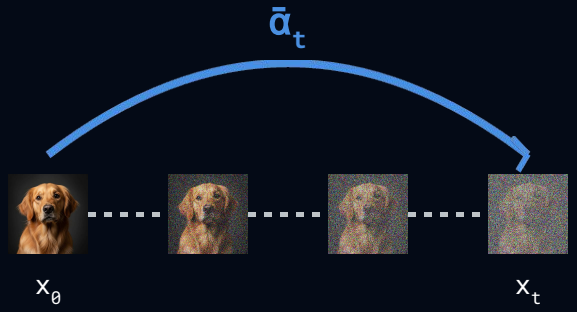
The Reparameterization Trick

Jumping directly to timestep t

Gaussian steps compose, so $q(x_t \mid x_0)$ has a closed form:

- **Definition:** $\bar{\alpha}_t = \prod_{s \le t} (1 - \beta_s)$, the fraction of signal variance kept up to step t.
- **Meaning:** $\sqrt{\bar{\alpha}_t}$ scales the clean image x_0; $\sqrt{(1 - \bar{\alpha}_t)}$ scales the noise ε.
- **Signal decay:** $\bar{\alpha}_t \to 0$ as $t \to T$, so $x_T \approx N(0, I)$.
- **Why it matters:** training can sample any x_t in one shot, without simulating the chain.

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$



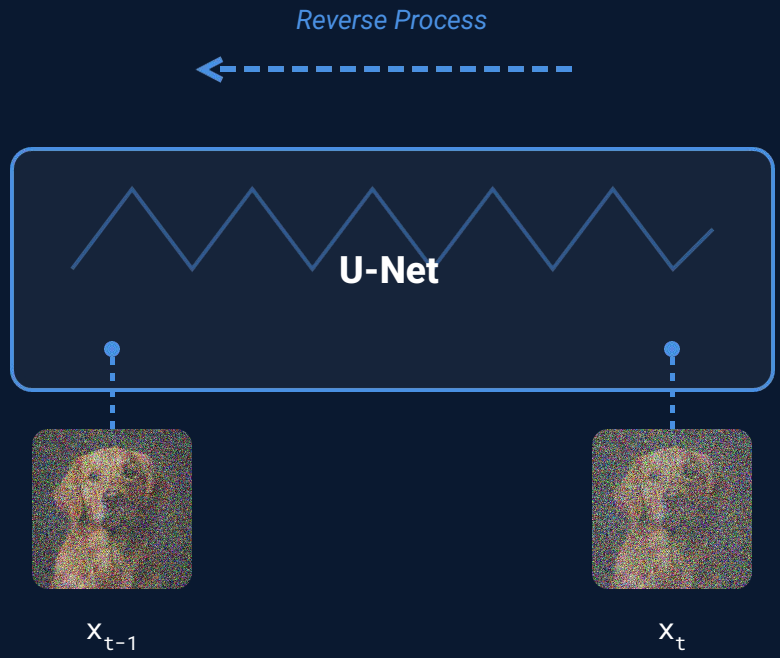
Direct Sampling with the Reparameterization Trick

The Reverse Process

The true reverse step is intractable. $q(x_{t-1} \mid x_t)$ depends on the unknown data distribution, so it has no closed form.

So the model learns it. A network (U-Net or Transformer) predicts each reverse step as a Gaussian:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1};\ \mu_\theta(x_t, t),\ \sigma_t^2 I\big)$$



Training Objective: ELBO → MSE

ELBO: maximizing this lower bound on log $p_\theta(x_0)$ means minimizing per-step KLs between $q(x_{t-1} \mid x_t, x_0)$, closed-form given x_0, and $p_\theta(x_{t-1} \mid x_t)$.

Match the means: both are Gaussians with fixed variance, so each KL only compares their means:

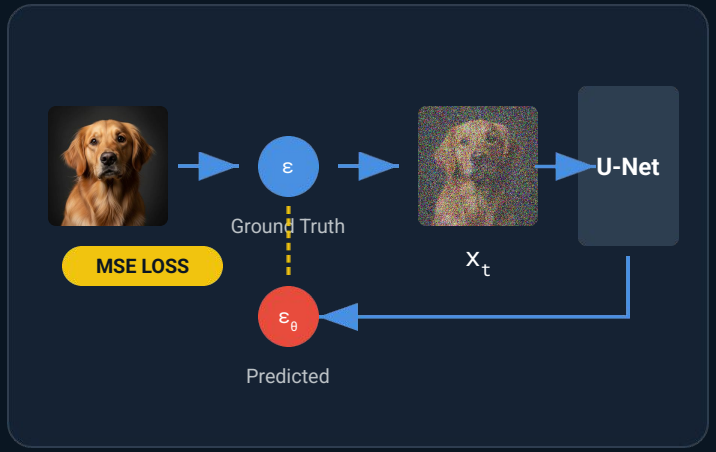
$$\mathrm{KL}_t = \frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t) \right\|^2 + C$$

Predict the noise: with $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon)/\sqrt{\bar{\alpha}_t}$, the target mean's only unknown is the noise ε. So a network ε_θ predicts it, and μ_θ copies the form:

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon \right) \quad \alpha_t = 1 - \beta_t$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

Result: each KL becomes $w_t \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2 + C$.



$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left\| \epsilon - \epsilon_\theta(x_t, t) \right\|^2$$

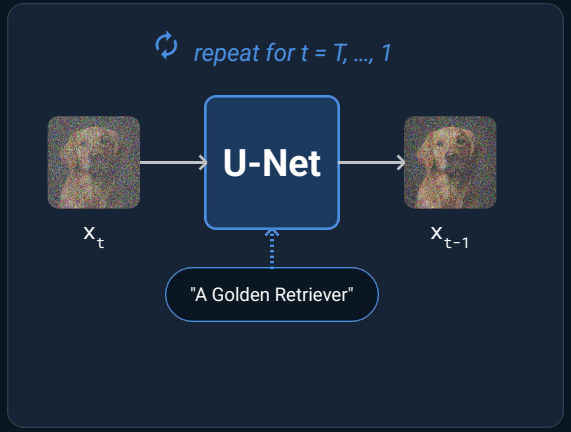
$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \quad t \sim \mathcal{U}\{1, \ldots, T\}$$

Inference (Sampling)

Start from pure noise: $x_T \sim N(0, I)$.

Denoise step by step, for t = T, …, 1: the network, given x_t, t and the prompt c, predicts the noise. That gives the mean μ_θ, and we add fresh noise $\sigma_t z$.

Each step is a Gaussian draw: a stochastic policy. One image costs T network calls.



$$x_{t-1} = \underbrace{\mu_\theta(x_t, t, c)}_{\text{network mean}} + \underbrace{\sigma_t z}_{\text{noise}}, \qquad z \sim \mathcal{N}(0, I)$$

The Limitation of Standard Paradigms

The Core Issue

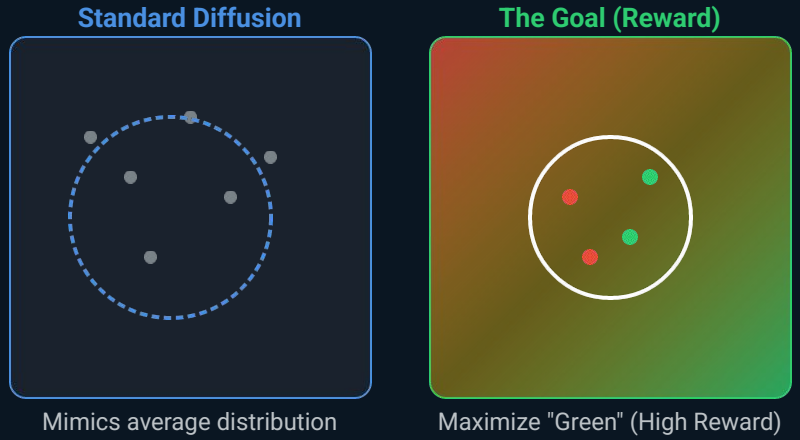
Despite the power of prompting, the foundation of this process is maximum-likelihood (supervised) training.

The Ceiling of Quality

Because the objective simply mimics data, conditioning mainly selects a subset of the training distribution. Going beyond the data (e.g., curated fine-tuning) stays heuristic until we optimize an explicit objective.

The Missing Link

The standard paradigm fundamentally does not optimize external rewards. Mimicking average past behavior is insufficient for safety constraints or complex preferences. RL optimizes the reward directly, actively seeking high-reward samples.





Section 03

The RL/MDP Formulation

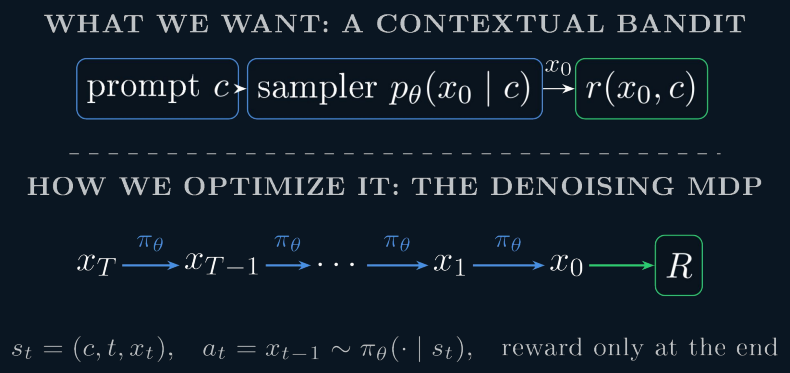
The Paradigm Shift

The agentic view: treat sampling as sequential decision-making, not only as density estimation.

Sequential actions: at each step the denoiser picks the next, less noisy sample x_{t-1}. The choice is stochastic (the $\sigma_t z$ term).

Terminal evaluation: only the final x_0 is scored, e.g. by an aesthetic model, a safety classifier, or a VLM judge.

Bandit vs. MDP: the task is a contextual bandit (prompt → image → reward). The MDP is a device that gives tractable per-step likelihoods for policy gradients.



Mapping Diffusion to an MDP

State $s_t = (c, t, x_t)$: prompt, timestep, noisy sample.

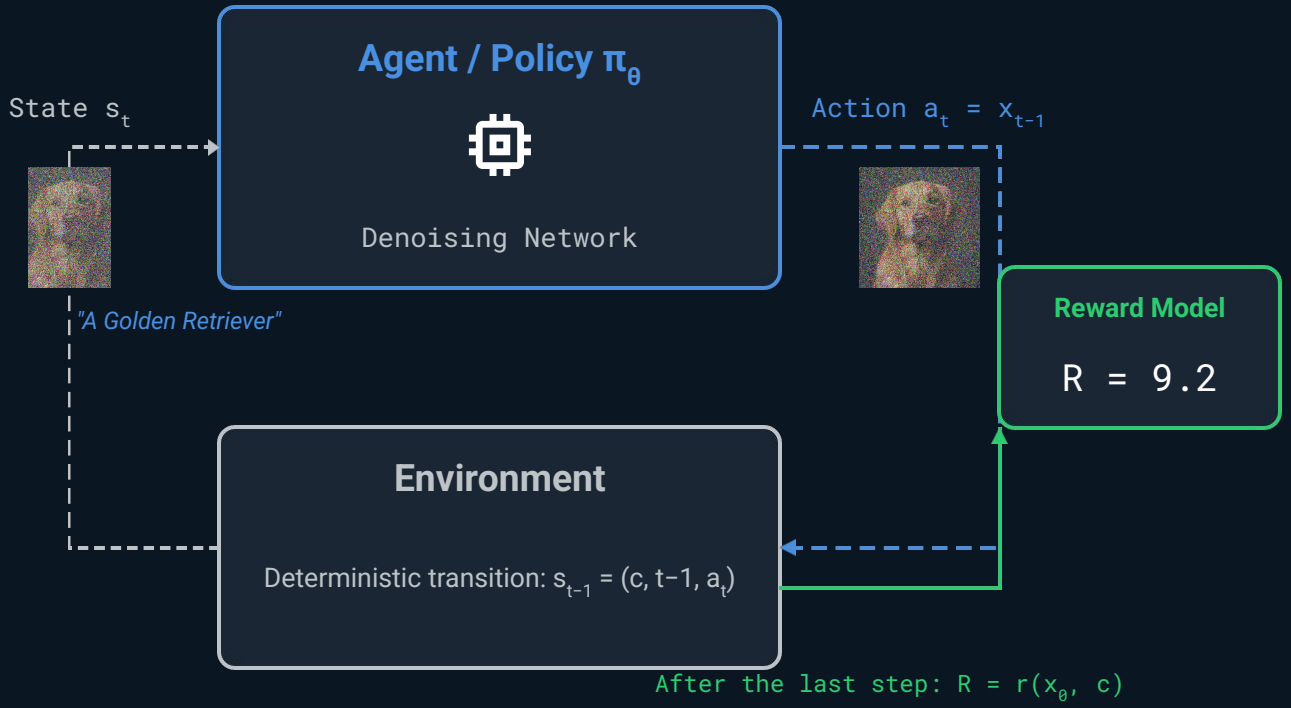
Action $a_t = x_{t-1}$: the next, less noisy sample.

Policy $\pi_\theta(a_t \mid s_t) = N(\mu_\theta(x_t, t, c), \sigma_t^2 I)$: one denoising step. ε_θ only sets the mean.

Transition: deterministic, $s_{t-1} = (c, t-1, a_t)$. All randomness is in ρ_0 and π_θ.

Initial state ρ_0: $c \sim p(c)$, $x_T \sim N(0, I)$.

Reward: $r(x_0, c)$ after the last step, 0 before.



One Objective Behind Every Method

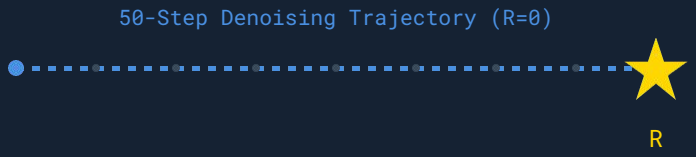
$$\max_{\theta} \; \mathbb{E}_{c}\Big[\mathbb{E}_{x_0\sim p_\theta(\cdot|c)}\, r(x_0,c) - \beta\,\mathrm{KL}\big(p_\theta(\cdot\mid c)\,\|\,p_{\mathrm{ref}}(\cdot\mid c)\big)\Big] \quad\Longrightarrow\quad p^{\star}(x_0\mid c) = \tfrac{1}{Z(c)}\,p_{\mathrm{ref}}(x_0\mid c)\,e^{r(x_0,c)/\beta}$$

Family	Methods (this lecture)	How it moves p_θ toward p*
Weighted likelihood	RWR, filtered fine-tuning; RL-D² FKL	Fit p_θ to samples reweighted by $e^{r/\beta}$ (or softmax(A/λ)) via the ELBO. Off-policy, stable.
Policy gradient	DDPO (β = 0), Flow-GRPO (β > 0); RL-D² RKL	Score-function gradients with per-step likelihood ratios on the denoising MDP. On-policy.
Closed-form reparameterization	Diffusion-DPO	Plug $r = \beta \log(p^*/p_{ref})$ + const into Bradley–Terry; fit offline on preference pairs.
Pathwise gradient	DRaFT, AlignProp, ReFL	Backpropagate a differentiable reward through (part of) the sampling chain.

The Two Massive RL Challenges

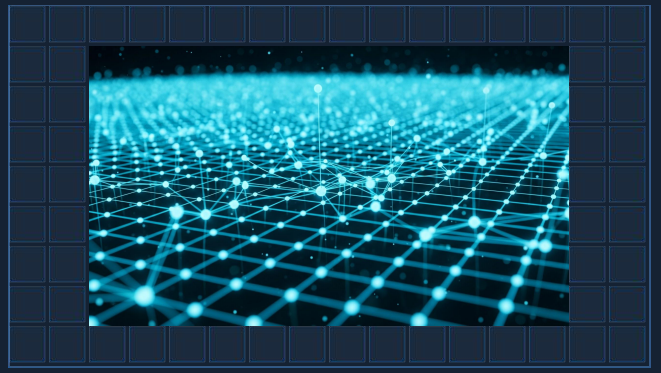
01. Extreme Sparsity

Reward arrives only after the last of ~50 steps (DDPO; Flow-GRPO trains with 10). Every step gets the same advantage, so per-step credit assignment is weak.



02. Massive Action Space

Each action is a whole latent x_{t-1} (SD 1.4: 4×64×64 = 16,384 dims). log π is analytic, but gradients are high-variance and per-step ratios are fragile.



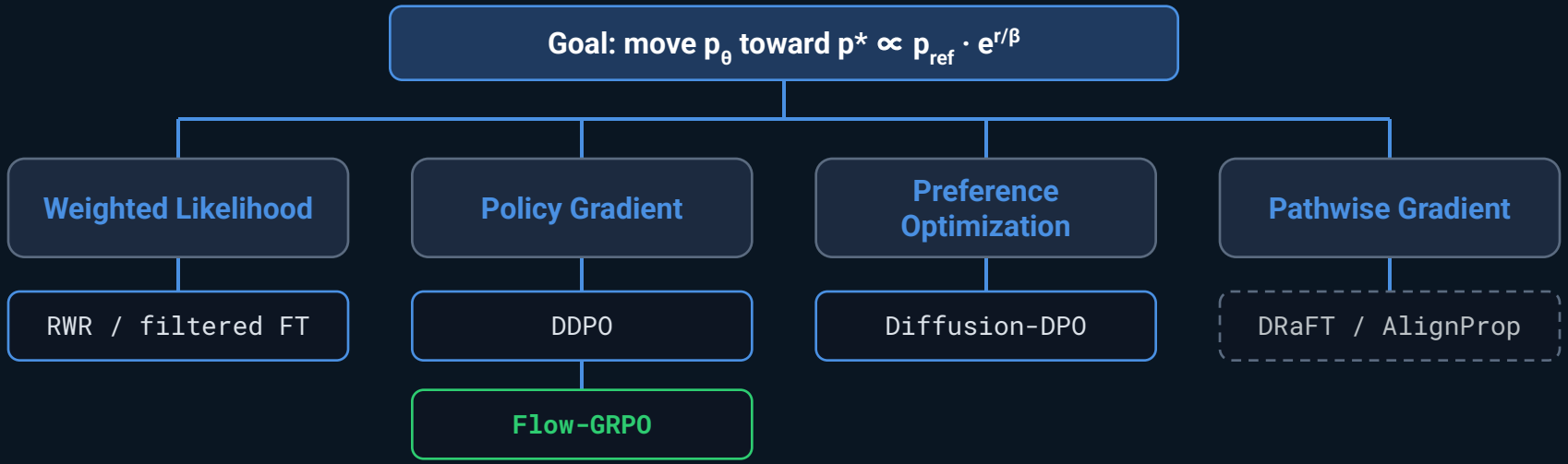
dim(A) = 4 × 64 × 64 (latent)



Section 04

RL Training Algorithms

Algorithmic Landscape Overview



Section 5 revisits the first two families for discrete diffusion policies: RL-D² FKL (weighted likelihood) vs. RKL (policy gradient).

Baseline: Reward-Weighted Regression (RWR)

Weighted maximum likelihood on the model's own samples

Each round: sample $x_0 \sim p_{old}(\cdot|c)$, score $r(x_0, c)$, then fit the reward-weighted diffusion loss:

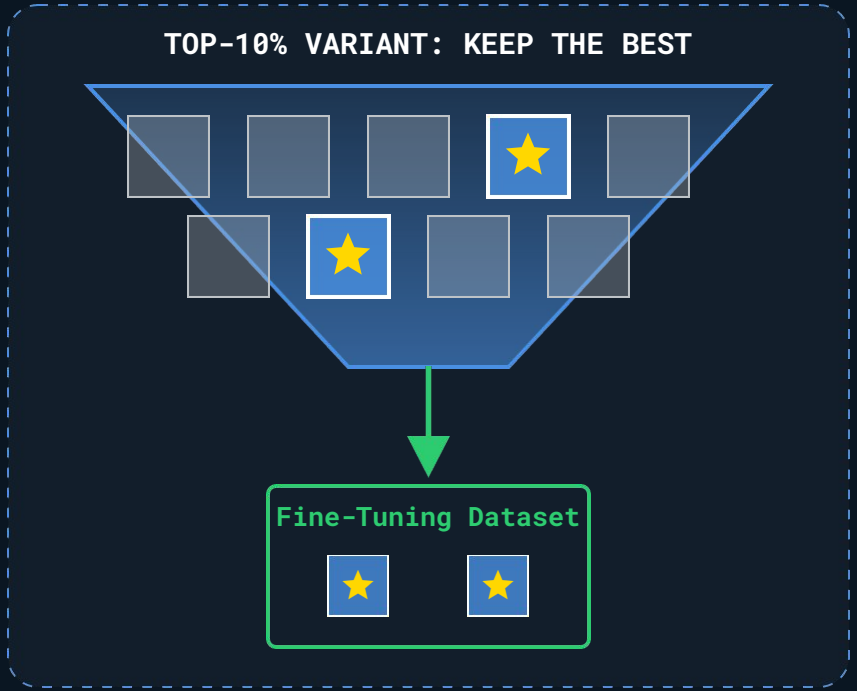
$$\max_\theta \; \mathbb{E}_{c,\; x_0 \sim p_{\text{old}}(\cdot|c)} \left[w(x_0, c) \cdot \text{ELBO}_\theta(x_0 \mid c) \right]$$

$$w \propto e^{r(x_0,c)/\beta} \quad \text{(soft)} \qquad \text{or} \qquad w = \mathbf{1}[r \geq \tau] \quad \text{(top 10\%)}$$

Link to p*: with soft weights the best fit is $\propto p_{old} \cdot e^{r/\beta}$: one KL-regularized improvement step.

Top-10% variant = repeated filtered fine-tuning: simple, but it discards 90% of the samples.

Weakness: the ELBO stands in for $\log p_\theta$, so RWR optimizes the objective only approximately.



Peters & Schaal, ICML 2007; diffusion version and baselines: Black et al., ICLR 2024

DDPO (Denoising Diffusion Policy Optimization)

Objective: maximize the expected reward $J(\theta) = E[r(x_0, c)]$, with x_0 generated by the model. No KL term ($\beta = 0$).

Policy gradient on the denoising MDP: each denoising step is an action with an exact Gaussian log-likelihood. The reward arrives only after the last step.

No critic: the advantage is the reward standardized per prompt, $A = (r - \mu_c)/\sigma_c$, where μ_c and σ_c are the mean and std of recent rewards for prompt c.

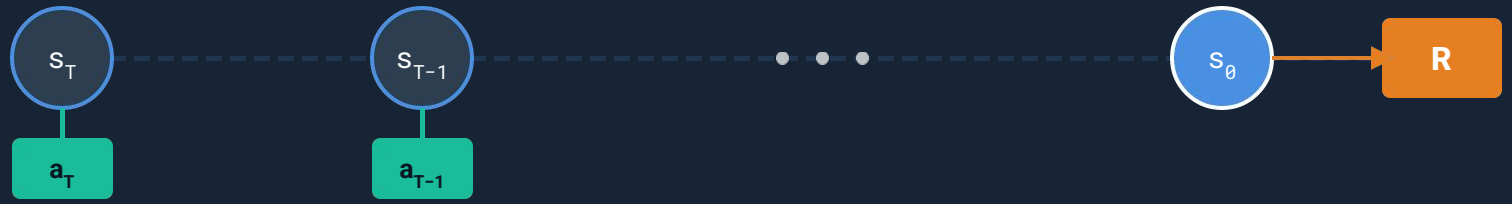
ONE DDPO ITERATION (REPEAT)

1. **Sample** 256 trajectories with π_θ (T = 50 steps); store every x_t and log π_θ.
2. **Score** each final image with $r(x_0, c)$. Nothing before the end.
3. **Normalize per prompt:** $A = (r - \mu_c) / \sigma_c$.
4. **Update** θ with a policy gradient summed over all T steps (two variants on the next slides).

Black, Janner, Du, Kostrikov & Levine, ICLR 2024 · arXiv:2305.13301

DDPO-SF: REINFORCE on Trajectories

ONE EPISODE = ONE DENOISING CHAIN



$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(x_{t-1} \mid x_t, c)\; A(x_0, c)\Big], \qquad A(x_0, c) = \frac{r(x_0, c) - \mu_c}{\sigma_c}$$

Roll out with the current policy

Store every (s_t, a_t, $\log \pi_\theta(a_t|s_t)$). The reward $r(x_0, c)$ is observed only at s_0. One gradient step per 256 samples.

One advantage for the whole chain

No critic and no discounting: all T steps get the same A, so there is no per-step credit assignment (the sparsity challenge).

Black et al., ICLR 2024 (DDPO-SF = REINFORCE, Williams 1992)

DDPO-IS: PPO-Style Sample Reuse

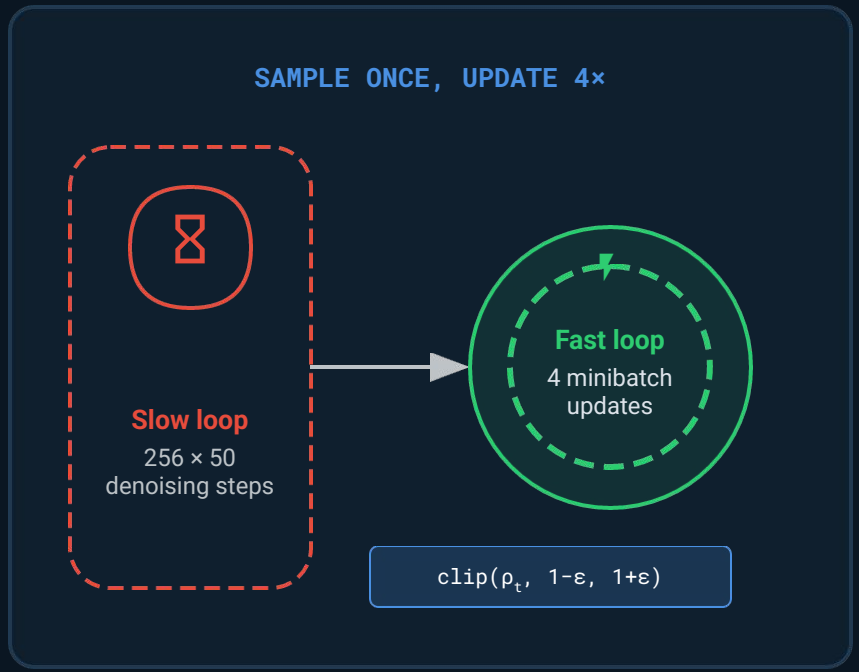
The bottleneck: one batch costs 256 × 50 denoising steps, yet DDPO-SF uses it for one gradient step.

The fix: reuse each batch for 4 minibatch updates, with per-step importance ratios ρ_t clipped as in PPO:

$$\max_{\theta} \mathbb{E}_{\pi_{\theta_{\text{old}}}}\left[\sum_{t=1}^{T} \min\left(\rho_t A,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\, A\right)\right]$$

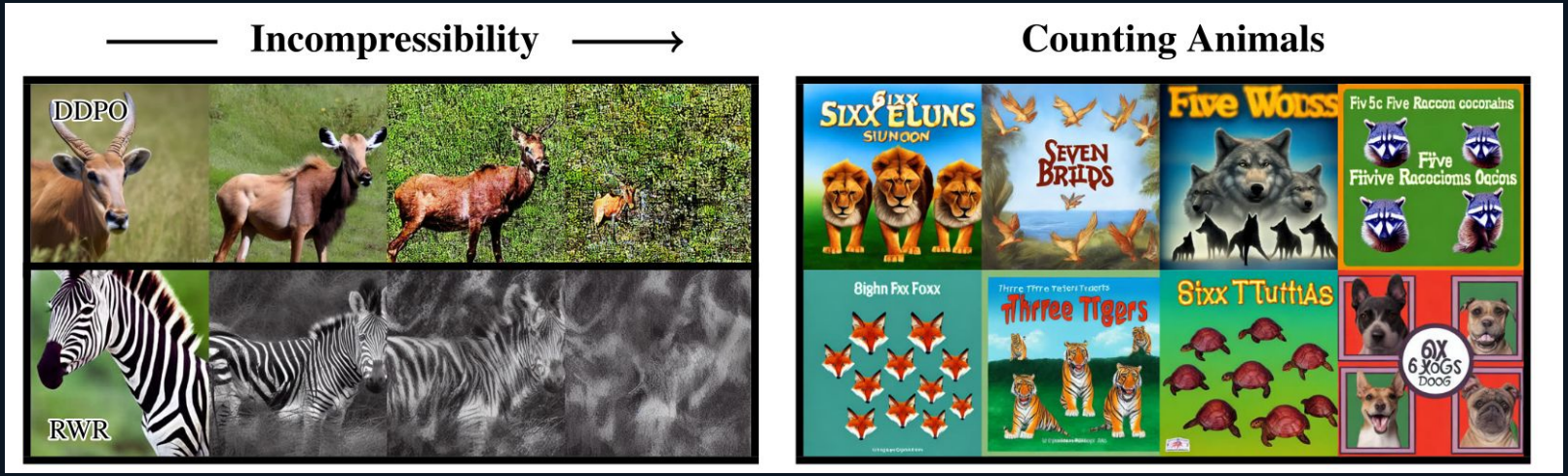
$$\rho_t = \frac{\pi_\theta(x_{t-1} \mid x_t, c)}{\pi_{\theta_{\text{old}}}(x_{t-1} \mid x_t, c)}, \qquad \varepsilon = 10^{-4}$$

Why such a tiny clip range (ε = 10^{-4})? The noise σ_t is small, so even a tiny change in the predicted mean moves ρ_t far from 1 (log ρ_t has std $\|\Delta\mu\|/\sigma_t$).



Black et al., ICLR 2024 (App. D); PPO: Schulman et al., 2017 · arXiv:1707.06347

Reward Hacking (Over-optimization)



DDPO (Black et al., 2024), Fig. 7. Left: optimizing incompressibility ends in noise. Right: the LLaVA counting reward is fooled by text ("sixx ttutttas" above eight turtles).

DDPO's remedy: early stopping (keep the last checkpoint before quality degrades). No KL term.

KL to the reference (β > 0): Flow-GRPO (Liu et al., 2025), GRPO-style RL for flow-matching models.

Why: the target $p^* \propto p_{ref}\, e^{r/\beta}$ collapses onto argmax r as $\beta \to 0$, exactly where the proxy reward is wrong.

Diffusion-DPO (Wallace et al., 2023)

Offline, from preferences: pairs where x^w is preferred over x^l for the same prompt. No reward model, no rollouts.

DPO in one line: the KL-regularized optimum gives $r = \beta \log(p^*/p_{ref}) + \beta \log Z(c)$. Plug it into Bradley–Terry, $P(x^w \text{ beats } x^l) = \sigma(r(x^w) - r(x^l))$: Z cancels, leaving a logistic loss on θ (right).

Diffusion twist: $\log p_\theta(x_0 \mid c)$ is intractable, so it is replaced by the ELBO, as in RWR. The loss then pushes ε_θ to denoise the winner better, and the loser worse, than the frozen reference.

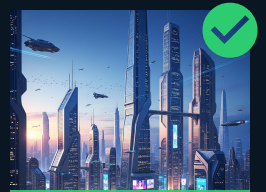


Image A (Preferred)

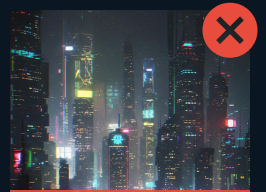


Image B (Rejected)

$$\mathcal{L}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{p_\theta(x^w \mid c)}{p_{\text{ref}}(x^w \mid c)} - \beta \log \frac{p_\theta(x^l \mid c)}{p_{\text{ref}}(x^l \mid c)}\right)\right]$$

Implicit reward: no reward model, no rollouts

Wallace et al., 2023 · arXiv:2311.12908; DPO: Rafailov et al., 2023 · arXiv:2305.18290

Online RL vs. Diffusion-DPO

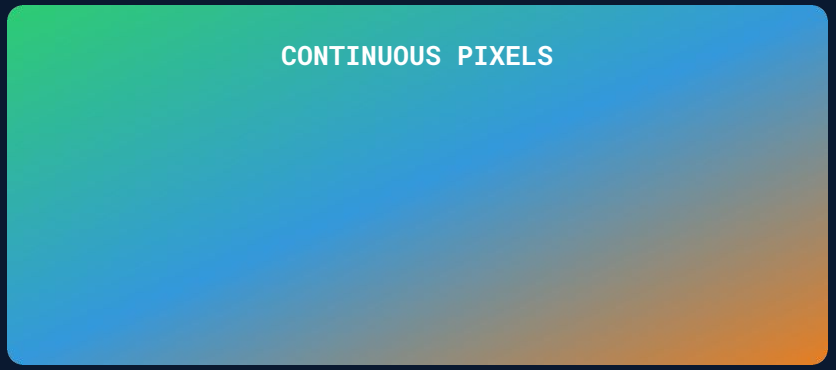
FEATURE	ONLINE RL (DDPO, FLOW-GRPO)	DIFFUSION-DPO (OFFLINE)
Data & reward	Fresh samples from the current model, scored by any reward, even black-box (JPEG size, OCR, VLM judge).	A fixed dataset of human preference pairs; the reward stays implicit.
Compute	Rollouts of T denoising steps per sample, plus reward calls.	No rollouts: one random timestep per pair, like ordinary fine-tuning.
Exploration & risk	Can find samples beyond the data, and can hack the reward.	Cannot explore beyond the preference data, but stays close to it.

Section 05

Discrete Diffusion Models



What if the Data is Discrete?





The Data

Text tokens, graph edges, DNA bases, and combinatorial actions in RL.

The Problem

Gaussian noise needs a continuous space: "cat" + 0.1·ε means nothing. One can diffuse embeddings (Diffusion-LM), but corrupting tokens directly is more natural.

Key Applications

Text and code generation, planning over discrete states, molecule and DNA design.

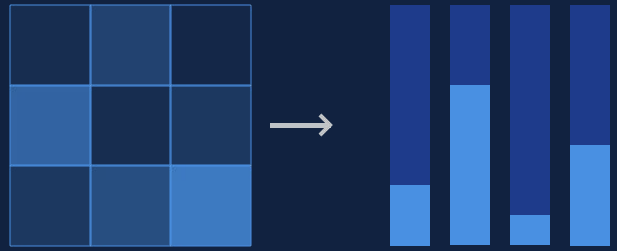
Discrete Denoising Diffusion (D3PMs)

D3PM: replace Gaussian noise with a Markov transition matrix Q_t over the vocabulary.

Tokens are one-hot row vectors. Each forward step multiplies by Q_t, so the t-step marginal is closed-form too:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ p = x_{t-1} Q_t\big)$$
$$q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ p = x_0 \bar{Q}_t\big), \quad \bar{Q}_t = Q_1 Q_2 \cdots Q_t$$

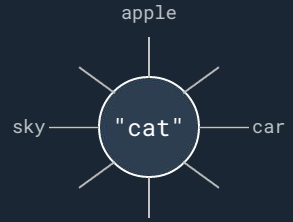
MARKOV TRANSITION MATRIX Q_t



$$p_t = p_{t-1}\ Q_t$$

Types of Discrete Corruption

UNIFORM



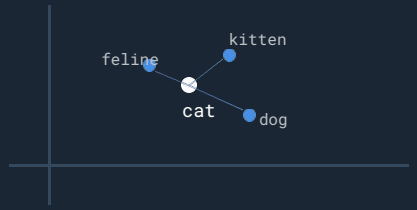
With probability β_t, a token is resampled uniformly from the vocabulary. The chain converges to the uniform distribution.

ABSORBING STATE



With probability β_t, a token becomes [MASK] and stays masked; unmasked tokens are copied through. Close to BERT's masked LM. Used by MDLM, LLaDA and RL-D².

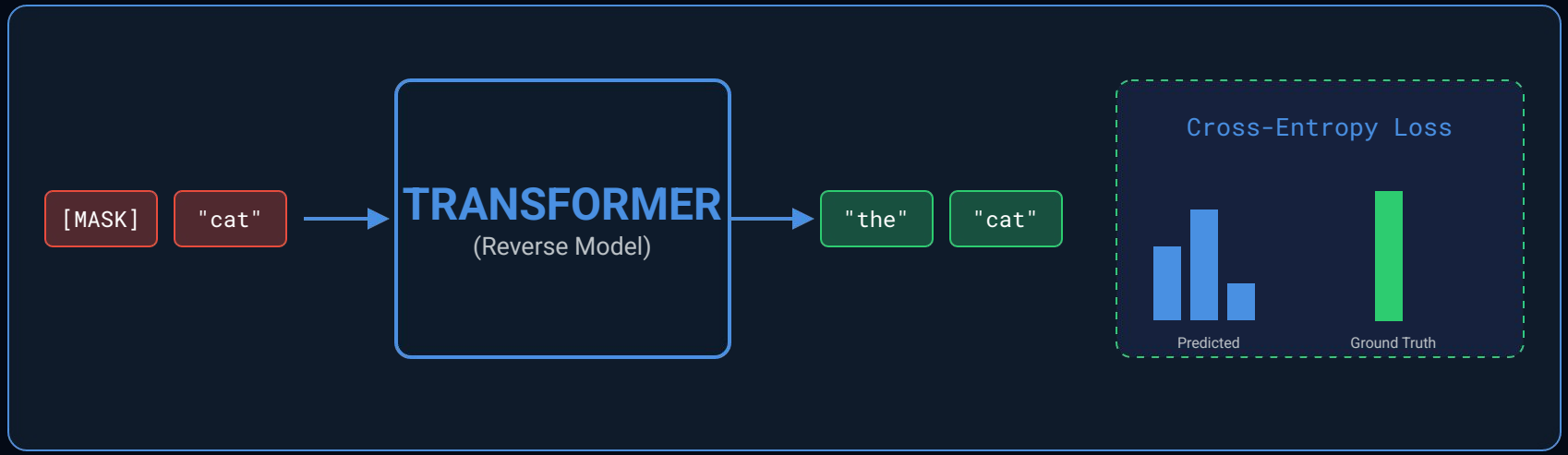
EMBEDDING DISTANCE



Tokens mostly flip to their nearest neighbors in an embedding space, so the corruption stays semantically local.

Austin et al., 2021 (D3PM); masked diffusion: Sahoo et al., 2024 (MDLM) · arXiv:2406.07524

The Discrete Reverse Process & Objective



The Process

A Transformer predicts the clean tokens x_0 from the corrupted x_t. The reverse step plugs this prediction into the closed-form posterior $q(x_{t-1} \mid x_t, x_0)$.

The Objective

For masked diffusion the ELBO becomes a weighted cross-entropy on the masked positions (BERT-like, with a noise schedule). It replaces the MSE of continuous diffusion.

Austin et al., 2021; Sahoo et al., 2024 (MDLM); Shi et al., 2024

RL for Diffusion vs. Diffusion for RL

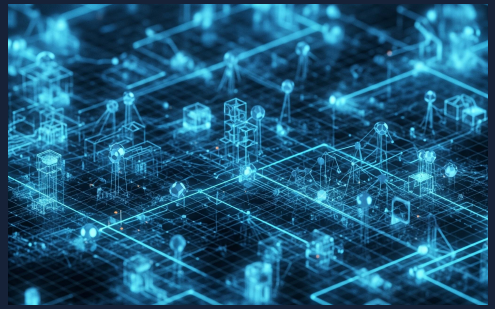
	RL FOR DIFFUSION	DIFFUSION FOR RL
Real problem	Contextual bandit: prompt $c \rightarrow$ sample $x_0 \rightarrow$ reward $r(x_0, c)$	Environment MDP: state $s \rightarrow$ action $a \rightarrow$ reward, next state
Diffusion model	The generator we fine-tune	The policy $\pi_\theta(a \mid s)$: denoising produces the action
Denoising chain	An MDP we build to get per-step likelihoods; its horizon is a design choice	Nested inside every environment step (augmented MDP with states (s, a^n))
Examples	DDPO, Flow-GRPO, Diffusion-DPO	DPPO (continuous; fine-tunes a BC-trained Diffusion Policy); RL-D^2 (discrete)

Why diffusion policies for combinatorial actions? An action $a = (a_1, \ldots, a_K)$ has $|A|^K$ values: too many for a softmax head. Autoregressive heads impose an arbitrary order and need K decoding steps; diffusion denoises all components in parallel and can use fewer than K steps.

Diffusion Policy: Chi et al., 2023 · arXiv:2303.04137; DPPO: Ren et al., 2024 · arXiv:2409.00588

RL-D²: RL with Discrete Diffusion Policies

COMBINATORIAL DISCRETE ACTION SPACE



Policy Mirror Descent (PMD)

TARGET DISTRIBUTION



$$\pi^{k}_{\mathrm{MD}}(a \mid s) = \frac{\pi_k(a \mid s)\, e^{A^{\pi_k}(s,a)/\lambda}}{Z_k(s)}$$

The Challenge

Actions $a = (a_1, \ldots, a_K) \in A^K$: macro-actions, joint multi-agent actions, slates. $|A|^K$ is far too many for a softmax policy head.

The Policy

A masked discrete diffusion model $\pi_\theta(a \mid s)$ generates all K components in parallel. Training alternates policy evaluation (advantages under π_k) and improvement.

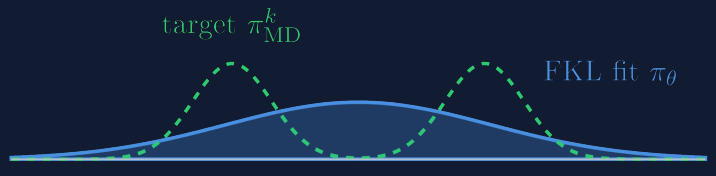
The Improvement Step

PMD target $\pi^k_{\mathrm{MD}} \propto \pi_k\, e^{A/\lambda}$: the KL tilt behind p*, per state and per iteration. Then fit π_θ to it with FKL or RKL (next slides).

Ma, Nabati, Rosenberg, et al., 2025 · arXiv:2509.22963; PMD: Shani et al., 2020 · arXiv:1909.02769; Tomar et al., 2020; Lan, 2023

RL-D² Option 1: Forward KL (FKL)

BIMODAL TARGET VS. FKL POLICY



Mass-covering ("mean-seeking")

Tractable FKL loss (weighted ELBO)

$$\mathcal{L}_{\mathrm{FKL}}(\theta) = \mathbb{E}_{s\sim\mathcal{D},\ \hat{\mathcal{A}}_s\sim\pi_k}\Big[-\sum_{a^0\in\hat{\mathcal{A}}_s} w(a^0)\,\mathcal{L}_{\mathrm{ELBO}}(a^0, s;\theta)\Big]$$

$$(1)\quad w(a^0) = \frac{e^{A^{\pi_k}(s,a^0)/\lambda}}{\sum_{a\in\hat{\mathcal{A}}_s} e^{A^{\pi_k}(s,a)/\lambda}} \quad \text{(self-normalized)}$$

$$(2)\quad \mathcal{L}_{\mathrm{ELBO}}(a^0, s;\theta) \ \le\ \log \pi_\theta(a^0 \mid s)$$

The Role of FKL

$KL(\pi^k_{MD} \| \pi_\theta)$ is mass-covering: π_θ keeps mass on every high-value mode. It is weighted maximum likelihood: RWR with advantages.

Two Intractable Pieces

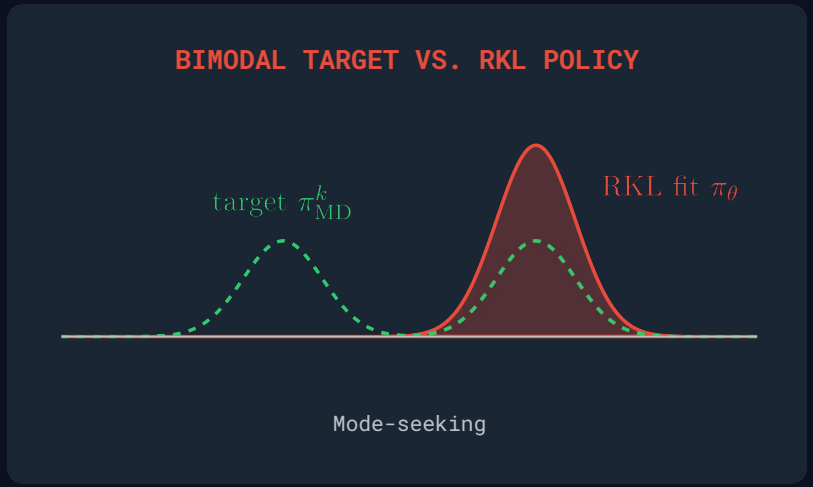
(1) The normalizer of π_k $e^{A/\lambda}$ over $|A|^K$ actions. (2) $\log \pi_\theta(a \mid s)$ of a diffusion policy.

The Fix

(1) Softmax over candidates $\hat{A}_s$ sampled from π_k. (2) Replace $\log \pi_\theta$ by its ELBO. Off-policy data can be reused.

Ma, Nabati, Rosenberg, et al., 2025 (Sec. 4) · arXiv:2509.22963

RL-D² Option 2: Reverse KL (RKL)



Single-step ratio (augmented MDP)

$$\eta\big((s,a^n),a^{n-1};\theta\big)=\frac{p_\theta(a^{n-1}\mid a^n,s)}{p_k(a^{n-1}\mid a^n,s)}$$

$$A^{\pi_k}\big((s,a^n),a^{n-1}\big)\ \triangleq\ A^{\pi_k}(s,a^0)$$

maximize a PPO-clipped surrogate in η

The Role of RKL

$KL(\pi_\theta \| \pi^k_{MD})$ is mode-seeking. Minimizing it is exactly the PMD step: maximize the expected advantage minus $\lambda \cdot KL(\pi_\theta \| \pi_k)$.

Likelihood via Denoising

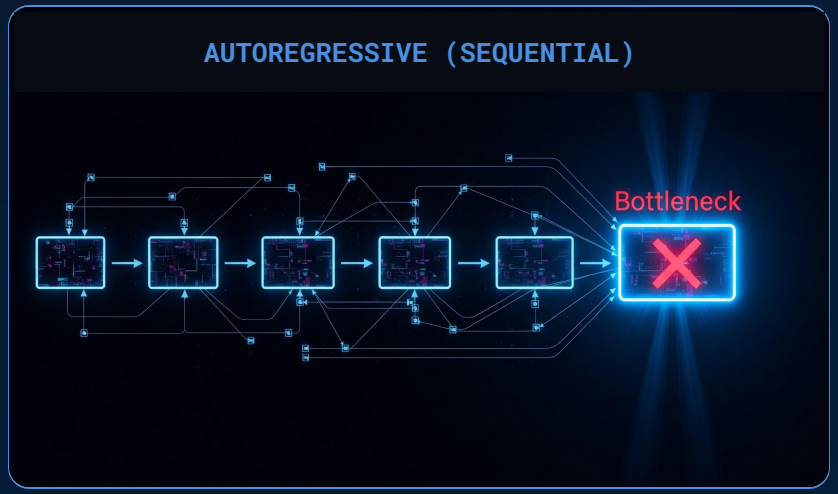
$\pi_\theta(a \mid s)$ is intractable, so treat each denoising step as an action, as in DDPO and DPPO; every step gets the final action's advantage.

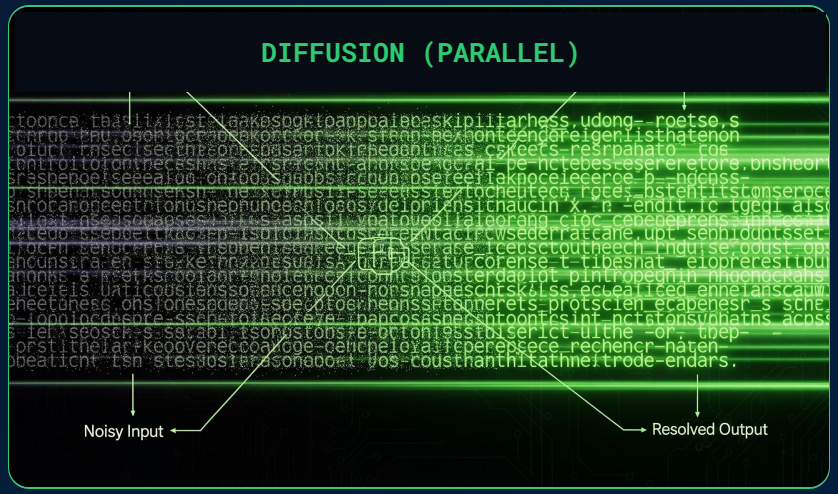
Stabilization

PPO-style clipping of η. The augmented MDP adds variance (RKL prefers large batches), but no ELBO bound is needed.

Ma, Nabati, Rosenberg, et al., 2025 (Sec. 4) · arXiv:2509.22963; augmented MDP: Ren et al., 2024 (DPPO)

The Future: Diffusion LLMs





The Premise

Parallel decoding: many tokens per forward pass. Reported speeds: Gemini Diffusion 1,479 tok/s (excl. 0.84 s overhead); Mercury Coder Mini 1,109 tok/s on an H100.

The Reality

Every pass attends over the whole sequence (no standard KV cache), and quality drops with too few steps. RL for dLLMs meets the same intractable likelihood as RL-D².

Gemini Diffusion: Google DeepMind, 2025; Mercury: Inception Labs, 2025 · arXiv:2506.17298; LLaDA 8B: Nie et al., 2025 · arXiv:2502.09992

Summary of the Lecture

1. Diffusion basics
2. Denoising MDP + KL
3. Algorithms
4. Diffusion policies

One objective, one trick (exact per-step likelihoods), several algorithms.

Diffusion basics: generation is a chain of small Gaussian denoising steps; training regresses the added noise (a simplified ELBO).

Denoising MDP + KL: each step is an action with an exact log-likelihood, so RL applies. We maximize reward with a KL anchor; the optimum is $p^* \propto p_{ref}\, e^{r/\beta}$.

Algorithms: different routes to p^*: reward-weighted likelihood (RWR), policy gradients (DDPO), preferences (Diffusion-DPO). Reward hacking is the main failure mode; the KL anchor or early stopping limits it.

Diffusion policies: with combinatorial actions, a discrete diffusion model can be the policy itself; RL-D^2 trains it with policy mirror descent, using a forward-KL (weighted likelihood) or reverse-KL (policy gradient) step.

Thank You

Questions?

Come work with us at **Google Research** on
Reinforcement Learning for LLMs and Diffusion Models.

